

Projektdokumentation

AEROGUARD - Intelligentes Raumluftüberwachungssystem

SoSe 2026

Ostbayerische Technische Hochschule Regensburg
Fakultät für Mathematik und Informatik

Themengebiet: Connected Home
Studienfach: DAPI – Ausgewählte Projekte der Informatik

Projektmitwirkende: **Prince Napoleon Nzeko Wagueu**

Betreuer: **Prof. Dr. Marcus Kucera, Dr. ing. Andrei Földi**

Abgabedatum: 20.06.2026

Inhaltsverzeichnis

1. Einleitung

- 1.1. Projektidee & Motivation
- 1.2. Zielsetzung

2. Technischer Überblick

- 2.1. Systemarchitektur
- 2.2. Hardware-Technologien
- 2.3. Kommunikationsprotokolle
- 2.4. Software-Technologien

3. Hardware

- 3.1. Systemaufbau
- 3.2. Sensoren & Hardwarekomponenten

4. Software

- 4.1. ESP32-Programmierung
- 4.2. Python-Backend
- 4.3. Web-Dashboard

5. Problemen und deren Lösungen

6. Inbetriebnahme & Deployment (Vom Quellcode zur Laufenden Umgebung)

7. Fazit & Ausblick

8. Stundenliste

1. Einleitung

1.1. Projektidee & Motivation

Die Qualität der Innenraumluft hat einen direkten Einfluss auf die menschliche Gesundheit und Konzentrationsfähigkeit, wobei insbesondere erhöhte CO₂-Werte rasch zu Müdigkeit führen und das Übertragungsrisiko von Viren steigern. Herkömmliche CO₂-Ampeln reagieren jedoch erst nach dem Überschreiten von Grenzwerten und vernachlässigen sowohl den zukünftigen Belastungstrend als auch die tatsächliche Anwesenheit von Personen, was oft zu unnötigen Energieverlusten durch falsches Lüften führt.

Aus dieser Problemstellung heraus entstand die Idee für AeroGuard: Ein intelligenter, vorausschauender Raumluf-Manager, der durch die Kombination aus photoakustischer CO₂-Sensorik, präziser mmWave-Radar-Präsenzerkennung und einem prädiktiven Backend eine proaktive Handlungsempfehlung liefert, noch bevor die Luftqualität in einen kritischen Bereich absinkt.

1.2. Zielsetzung

Das Projekt **AeroGuard** zielt darauf ab, ein energieeffizientes IoT-System zur Überwachung der Luftqualität in Vorlesungsräumen im Zusammenhang mit der Raumbesetzung zu entwickeln.

2. Technischer Überblick

2.1. Systemarchitektur

Wie in Abbildung 2 dargestellt, lassen sich aus dem Flussdiagramm (Abbildung 1) die grundlegenden Aufgaben der einzelnen Komponenten ableiten. Das intelligente Raumlufüberwachungssystem **AeroGuard** basiert auf einer modularen **3-Schichten-Architektur**, die eine effiziente Brücke zwischen der physischen Datenerfassung und einer dynamischen Benutzerinteraktionsschicht schlägt.

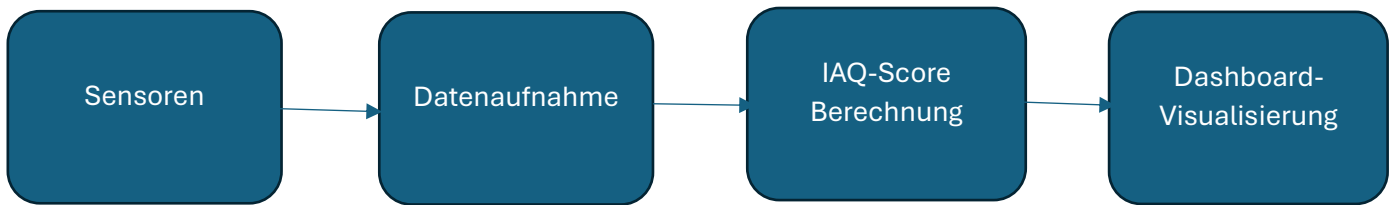
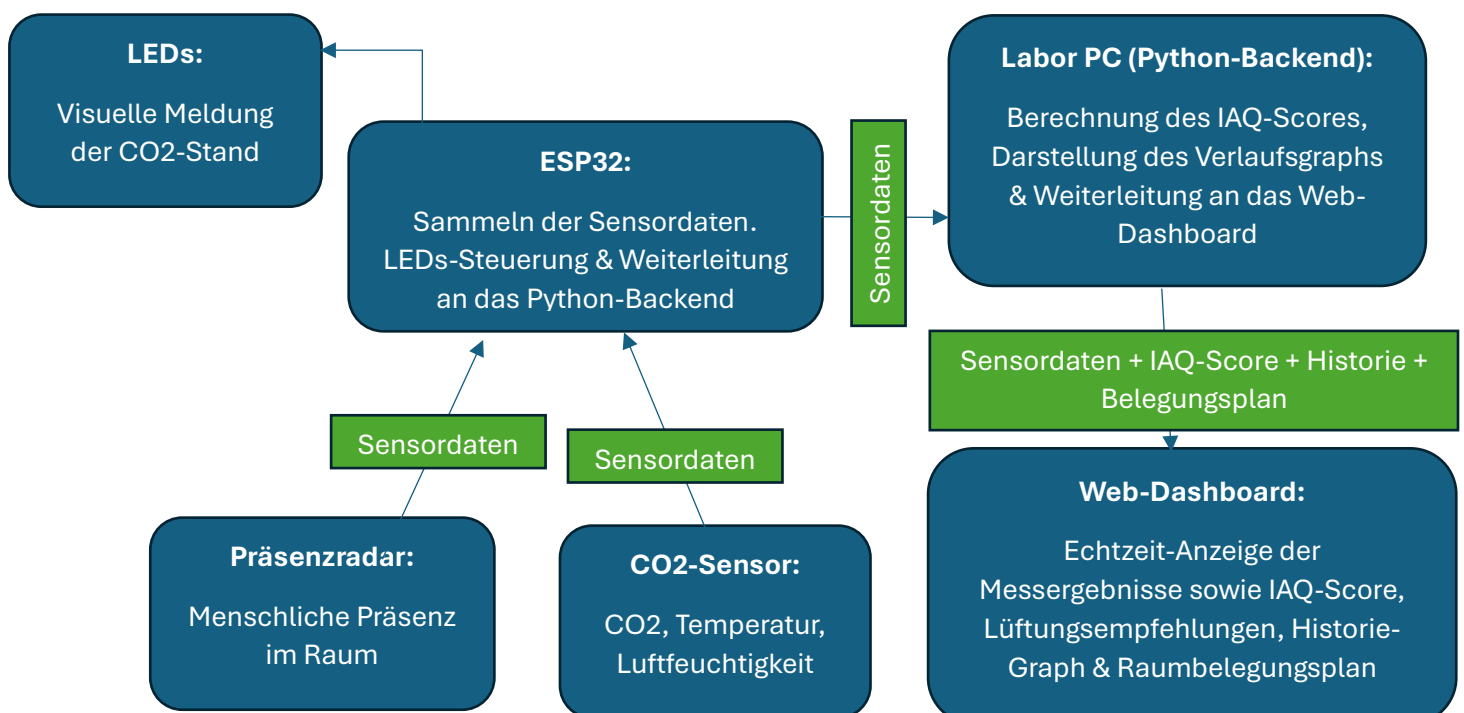


Abbildung 1: Datenflussdiagramm

Die Architektur teilt sich in folgende funktionale Ebenen auf:

- **Die Datenerfassungs- und Aktuationsschicht (Hardware-Schicht):** Der ESP32-Microcontroller agiert als physisches Gateway. Er fragt kontinuierlich die Sensordaten ab, steuert die lokale LED-Statusampel und leitet die Rohdaten drahtlos weiter.
- **Die Vermittlungs- und Persistenzschicht (Zentrales Backend):** Ein zentraler MQTT-Broker empfängt die Datenpakete. Dahinter verarbeitet ein FastAPI-Server die Payloads, berechnet den Luftqualitätsindex (IAQ) und speichert die bereinigten Werte in einer SQLite-Datenbank.
- **Die Präsentationsschicht (Web-Dashboard):** Ein modernes Web-Dashboard visualisiert die Messergebnisse in Echtzeit, gibt vorausschauende Lüftungsempfehlungen an den Endnutzer, stellt eine graphische Historie der Werte dar und verwaltet Raumbelegungspläne.

Abbildung 2: Aufgabenverteilung auf die einzelnen Komponenten



2.2. Hardware-Technologien

In diesem Projekt wurden folgende Hardwarekomponenten verwendet:

Bauteile	Bezeichnung / Typ	Funktion im Projekt
Microcontroller	ESP-32	Sammeln von Sensorwerten & LEDs-Steuerung
CO2-Sensor	Sensirion-SCD41	Messung von CO2-gehalt, Temperatur & Luftfeuchtigkeit
Radar für menschliche Präsenz	LD2410C	Erkennung der menschlichen Anwesenheit (auch völlig unbeweglich)
LEDs	Grün, Rot & Gelb	Visuelle Signalisierung der Luftqualität
Vorwiderstände	307 Ohm	Schützen der LEDs vor Überstrom & Zerstörung

2.3. Kommunikationsprotokolle

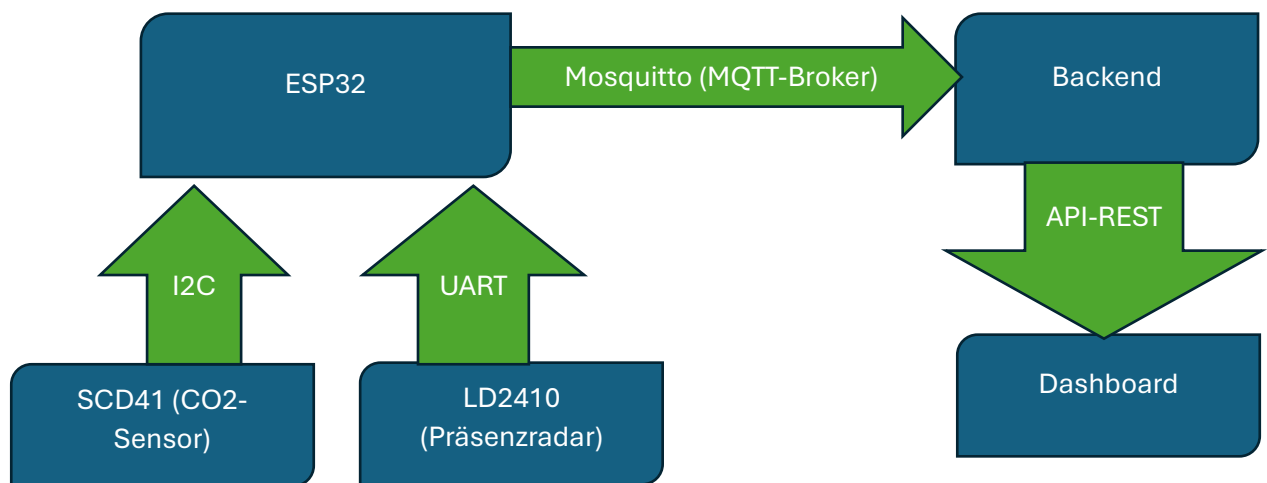
Die Interkonnektivität des Gesamtsystems beruht auf dem Zusammenspiel von Hardware-Bussen, leichtgewichtigen IoT-Protokollen und Standard-Web-APIs:

- **I2C-Protokoll (Inter-Integrated Circuit):** für die Kurzstrecken-Kommunikation zwischen dem ESP32 und dem SCD41. Es erlaubt die serielle Übertragung von Daten über nur zwei physische Leitungen (SDA/SCL).
- **UART-Protokoll (Universal Asynchronous Receiver-Transmitter):** Serielle Punkt-zu-Punkt-Verbindung zur Steuerung und zum Auslesen des LD2410C-Radars durch den ESP32
- **MQTT-Protokoll (Message Queuing Telemetry Transport):** lokal betrieben über den Broker **Mosquitto**. MQTT wurde gezielt gegenüber HTTP bevorzugt, da es durch sein **Publish/Subscribe-**

Modell eine asynchrone Echtzeit-Übertragung ermöglicht. Die Nachrichtenköpfe (Header) sind klein (nur wenige Bytes), was den Energie- und Rechenaufwand des ESP32 minimiert.

- **REST-API (HTTP GET / POST / DELETE):** Das Frontend kommuniziert über klassische HTTP-Anfragen mit dem FastAPI-Backend. Über GET werden die Sensordaten geladen, mittels POST und DELETE steuert der Nutzer die Eingabemaske des Raumbelegungsplaners direkt in der SQLite-Datenbank.

Abbildung 3: Verwendete Protokolle



2.4. Software-Technologien

Der Software-Stack wurde nach den Prinzipien der Modularität, Echtzeitfähigkeit und Wartbarkeit ausgewählt.

- **Firmware (C++ / Arduino IDE):** zur hardwarenahen Programmierung des ESP32 mit Nutzung bestimmter Bibliotheken (SensirionI2CScd4x, PubSubClient, Id2410) für eine stabile Sensoransprache und Netzwerkverbindung.
- **Backend-API (Python / FastAPI):** FastAPI wurde aufgrund seiner asynchronen Architektur und der automatischen Generierung von Schnittstellendokumentationen gewählt. Es fungiert als Rechenzentrum für den IAQ-Score und das mathematische Prädiktionsmodell.
- **Datenbank (SQLite):** zur permanenten Speicherung der komprimierten Umgebungsmesswerte sowie der

Raumbelegungsdaten. Eine leichtgewichtige, relationale Datei-Datenbank (aeroguard.db) wurde dafür erstellt.

- **Frontend-Dashboard (React.js & Vite):** React.js für eine komponentenbasierte, reaktive Benutzeroberfläche. **Vite** als Build-Tool für das System mit schnellen Ladezeiten. **Tailwind CSS** für das Styling, um ein vollständig responsives Design für Smartphones und PCs zu garantieren. Über die spezialisierte Bibliothek **Recharts** werden die zeitlichen Verläufe gerendert.

3. Hardware

3.1. Systemaufbau

Der physische Aufbau des AeroGuard-Systems folgt den Prinzipien des *Rapid Prototyping*. Ziel war es, ein funktionales und modulares System zu schaffen, das trotz Ressourcenbeschränkungen im Bereich der additiven Fertigung (3D-Druck) eine präzise Sensorerfassung und mechanische Stabilität gewährleistet.

3.1.1 Mechanische Konstruktion & Gehäusedesign

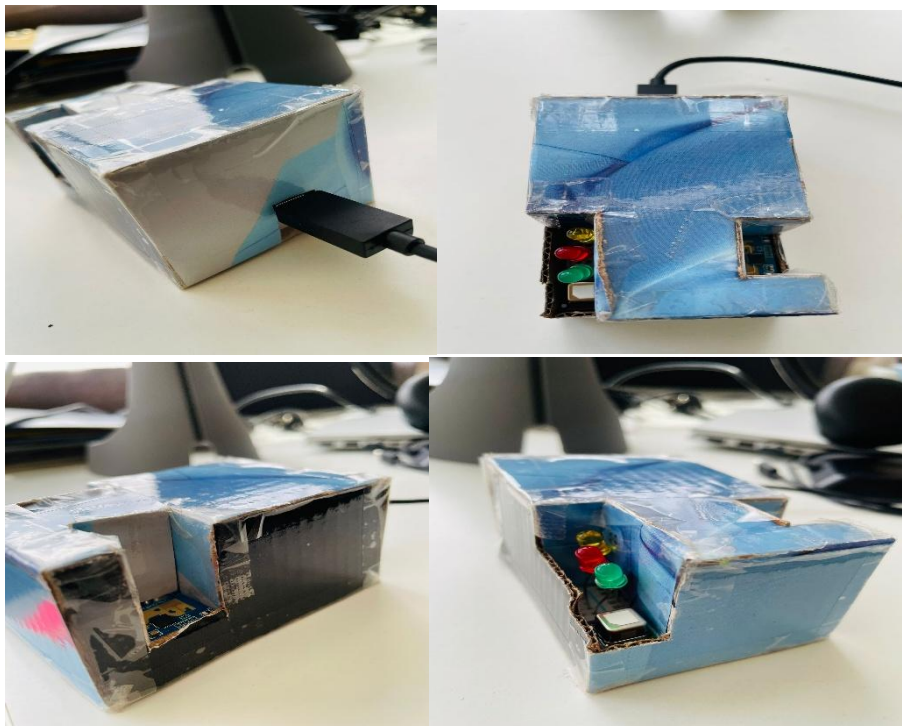


Abbildung 4: Systemaufbau

Aufgrund des temporären Verzichts auf ein 3D-gedrucktes wurde das finale Schutzgehäuse in präziser Handarbeit aus **Karton** gefertigt. Dieses Material bietet für die Prototypenphase entscheidende konstruktive Vorteile: Es besitzt ein geringes Eigengewicht, lässt sich leicht bearbeiten und weist eine ausreichende Steifigkeit auf, um die interne Elektronik vor äußeren mechanischen Einwirkungen zu schützen.

Um Messfehler und thermische Staus im Inneren zu verhindern, wurde das Gehäuse mit gezielten **Freischnitten** versehen:

- **SCD41 (CO2-Sensor):** Das Gehäuse spart den Bereich um den photoakustischen Sensor vollständig aus. Dies garantiert eine ungehinderte Luftzirkulation, sodass die Raumluft ohne Zeitverzögerung an die Messkammer gelangt.
- **LD2410C (Präsenzradar):** Obwohl hochfrequente Millimeterwellen dünne Materialien durchdringen können, wurde an der Frontseite eine exakte Aussparung integriert. Dies eliminiert jegliche Signalbefeuchtung oder Reflexion durch das Gehäusematerial und sichert die maximale Empfindlichkeit bei der Mikro-Präsenzerkennung.

3.1.2. Interne Elektronik-Montage & Signalwege

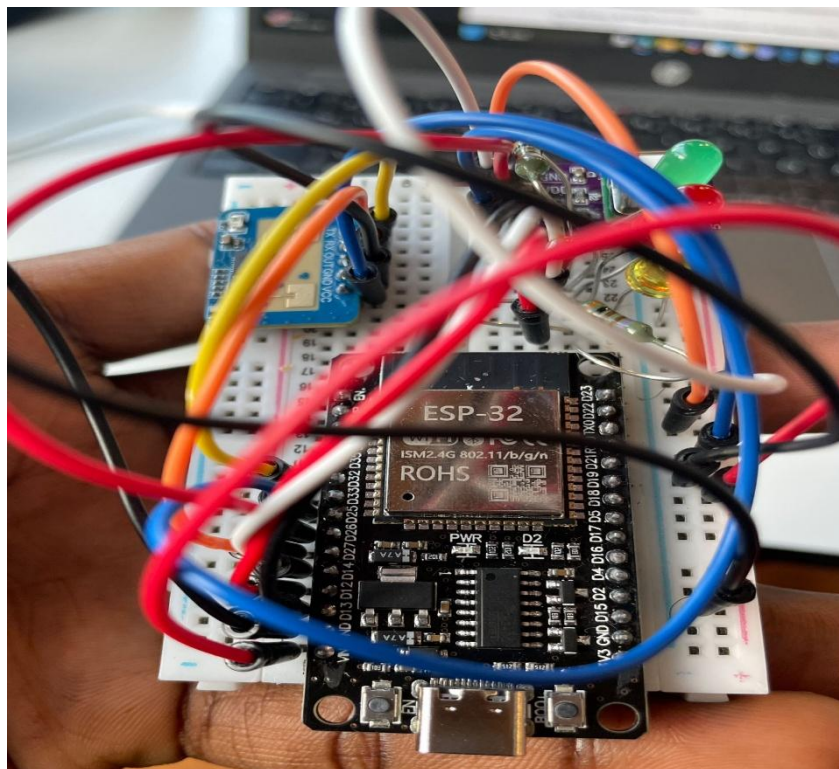


Abbildung 5: Elektronik-Montage

Die elektrische Verknüpfung und physische Fixierung der Komponenten im Gehäuseinnenraum erfolgten modular:

- **Das Breadboard-Prinzip:** Der ESP32-Microcontroller sowie die beiden Sensoren (SCD41 und LD2410C) sind direkt auf ein zentrales **Breadboard (Steckplatine)** gesteckt. Der Verzicht auf eine permanente Lötverbindung erhöht die Wartungsfreundlichkeit des Prototyps erheblich, da Komponenten zu Testzwecken jederzeit zerstörungsfrei ausgetauscht oder neu verkabelt werden können.
- **Kabelführung und Aktuatoren:** Die Verbindung der Status-LEDs sowie die Energieversorgung des ESP32 (via USB-C) werden über flexible Jumper-Kabel realisiert. Das Breadboard ist passgenau im Kartongehäuse fixiert, sodass die mechanischen Druckkräfte beim Einstecken des Versorgungskabels aufgefangen werden.

3.2. Sensoren & Hardwarekomponenten

3.2.1. Zentraler Microcontroller (ESP32 Dev Module)

Der ESP32 bildet das Gehirn des AeroGuard-Systems. Er ist ein leistungsstarker 32-Bit-Dual-Core-Microcontroller mit integriertem Wi-Fi- und Bluetooth-Modul. Wir haben uns für das ESP32 Dev Module entschieden, da es die notwendige Rechenleistung besitzt, um Sensorwerte asynchron auszulesen, die lokale LED-Ampel latenzfrei zu steuern und gleichzeitig eine permanente WLAN-Verbindung zum MQTT-Broker aufrechterhalten zu können.

Der ESP32 ist mittig auf dem Breadboard platziert und speist die Stromschienen der Steckplatine mit zwei Spannungsdomänen (3,3V und 5V). Über seine GPIO-Pins erfasst er die Daten des CO₂-Sensors über den I2C-Bus sowie des Päsensradars über eine serielle UART-Schnittstelle und steuert die digitalen Ausgänge der LEDs.

Das erste Testing erfolgte isoliert durch das Flashen eines einfachen "Blink"-Programms, um die Funktionsfähigkeit der GPIO-Pins zu überprüfen. Die WLAN-Verbindung wurde über den seriellen Monitor der Arduino IDE kalibriert, indem die Signalstärke in verschiedenen Entfernungen zum Router gemessen wurde, um Verbindungsabbrüche im Dauerbetrieb auszuschließen.



Abbildung 6: ESP32 Microcontroller

3.2.2. CO2-Sensor (SCD41)

Der Sensirion SCD41 ist ein hochpräziser Sensor zur Messung von Kohlenstoffdioxid (CO₂), Temperatur und relativer Luftfeuchtigkeit. Wir haben uns für den SCD41 entschieden, da er auf der innovativen **photoakustischen Spektroskopie (PAS)** basiert. Dabei strahlt eine integrierte Infrarot-Quelle Licht in eine winzige Messzelle. Die CO₂-Moleküle absorbieren dieses Licht und erzeugen durch periodische Erwärmung akustische Druckwellen, die von einem internen Mikrofon gemessen werden. Dies erlaubt eine extrem-kompakte Bauform bei maximaler Präzision.

Der Sensor ist auf dem Breadboard platziert und wird mit der 3,3V-Spannungsdomäne des ESP32 versorgt. Die Datenübertragung erfolgt digital über das I2C-Protokoll. Dazu wurden die Pins für Daten (SDA) an **GPIO 25** und Takt (SCL) an **GPIO 26** verdrahtet.

Das Testen des Sensors erfolgte zunächst mit der Beispiel-Library von Sensirion, um die korrekte I2C-Adressierung zu validieren. Der SCD41 besitzt eine integrierte automatische Selbstkalibrierung (ASC). Um korrekte Basiswerte zu garantieren, wurde das System für die Erstkalibrierung für ca. 30 Minuten an die frische Außenluft gestellt, da dort ein konstanter Referenzwert von ca. 400ppm CO₂ vorherrscht.

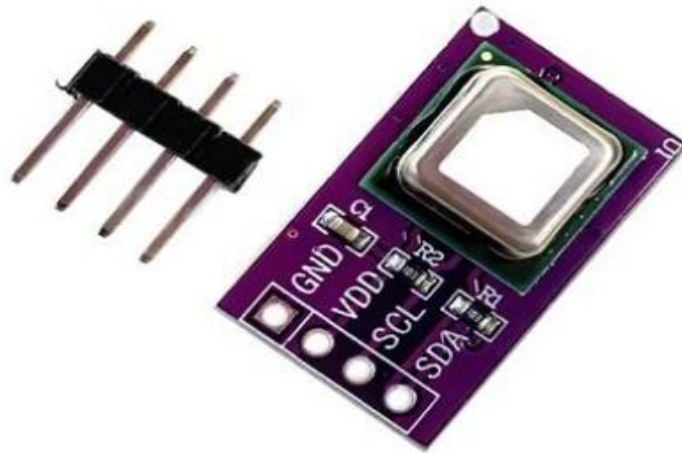


Abbildung 7: SCD41-Sensor

3.2.3. Präsenzradar (LD2410C)

Der LD2410C ist ein hochentwickelter menschlicher Präsenzstatussensor, der mit einem 24GHz-Millimeterwellen-Radar (mmWave) arbeitet. Wir haben uns für diesen Sensor entschieden, weil er im Gegensatz zu trägen PIR-Bewegungsmeldern in der Lage ist, Personen auch bei absoluter Immobilität (z. B. beim stillen Lesen oder Arbeiten am PC) zu erkennen. Er misst sowohl bewegte Ziele als auch statische Ziele durch die Erfassung von Mikrobewegungen (wie der Atembewegung des Brustkorbs).

Der Sensor ist am Rand des Breadboards aufgesteckt und wird für einen stabilen Betrieb mit der 5V-Schiene (VIN des ESP32) betrieben. Die Kommunikation mit dem Microcontroller erfolgt über eine serielle Punkt-zu-Punkt-Verbindung (UART). Der Sende-Pin (TX) des Radars ist mit **GPIO 32** (RX2 des ESP32) und der Empfangs-Pin (RX) mit **GPIO 27** (TX2 des ESP32) über Kreuz verbunden.

Das Testing erfolgte mithilfe der offiziellen Werkssoftware des Herstellers über Bluetooth, um die Empfindlichkeitsschwellen (Gates) der Radarantenne visuell zu überprüfen. Der Sensor wurde so kalibriert, dass er Personen in einem Radius von bis zu 6 Metern zuverlässig detektiert, Haustiere oder vorbeiziehende Luftströme von Klimaanlage jedoch herausgefiltert werden.

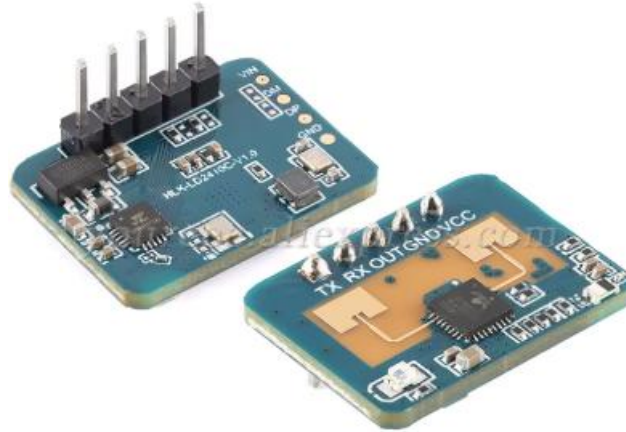


Abbildung 8: LD2410C Präsenzradar

3.2.4. LED-Statusampeln

Die LED-Ampel besteht aus drei separaten Leuchtdioden in den Farben Grün, Gelb und Rot. Sie dient als visuelles Echtzeit-Warnsystem direkt am Einsatzort. Wir haben uns für diese simple Dreifarben-Anzeige entschieden, damit Nutzer im Raum die Luftqualität (gesund, grenzwertig, kritisch) sofort intuitiv erfassen können, ohne ein digitales Dashboard aufrufen zu müssen.

Die drei LEDs sind in einer Reihe auf dem Breadboard montiert. Die kurzen Kathoden-Beine führen zur gemeinsamen Masse (GND). Den langen Anoden-Beinen ist jeweils ein **307 Omega-Vorwiderstand** vorgeschaltet, um den Stromfluss zu begrenzen. Die Steuerung erfolgt über digitale Signale (HIGH/LOW) ausfolgenden Ausgängen des ESP32: Grün an **GPIO 13**, Gelb an **GPIO 12** und Rot an **GPIO 14**.

Das Testing wurde mit einem einfachen sequenziellen Testskript durchgeführt, welches alle zwei Sekunden die Farbe wechselt. Die Schwellenwerte für die Ansteuerung wurden basierend auf internationalen Richtlinien kalibriert: Grün schaltet sich bei unter 800ppm ein, Gelb leuchtet exklusiv zwischen 800 und 1200ppm, und Rot signalisiert akuten Lüftungsbedarf bei über 1200ppm.



Abbildung 9: LED-Ampeln (grün, rot, gelb)

4. Software

4.1. ESP32-Programmierung

4.1.1. Sensorabfrage

Im ESP32-Programm werden verschiedene Sensoren mit individuell abgestimmten Methoden abgefragt. Die Datenverarbeitung erfolgt innerhalb der Hauptschleife *loop()*, wobei ein Timerinterrupt mit 2Hz (alle 500ms) den Hauptdatenversand steuert. Die eigentliche Sensorabfrage erfolgt im Triggerfall (*systemActive == true*), das heißt wenn eine menschliche Präsenz im Raum detektiert wird.

- Die menschliche Präsenz im Raum wird durch die Funktionen *radar.movingTargetDetected()* (Erkennung von Bewegung) und *radar.stationaryTargetDetected()* (Erkennung von statischer Präsenz) abgefragt. Die Ergebnisse werden logisch per ODER-Verknüpfung in der Variable *menschPraesenz* zusammengeführt, um den Systemstatus (*systemActive*) zu steuern.
- Die Raumluftdaten werden vom ESP32 über die Funktion *scd41.getDataReadyFlag(ready)* abgefragt, um das interne Status-Flag des Sensors zu prüfen. Die Variable *ready*, signalisiert, dass die photoakustische Messung abgeschlossen ist oder nicht.

4.1.2. CO2-Sensors (SCD41)

Initialisierung:

Innerhalb der Funktion *setup()* wird der serielle I2C-Datenbus über den Befehl *Wire.begin(25, 26)* auf den physischen Datenleitungen initialisiert. Der Sensor wird über das Objekt *scd41.begin(Wire)* angesprochen. Um undefinierte Zustände zu vermeiden, wird der Sensor zunächst mittels *scd41.stopPeriodicMeasurement()* gestoppt und anschließend über *scd41.startPeriodicMeasurement()* in den kontinuierlichen Messmodus versetzt.

Rohdatenerfassung:

Innerhalb des aktiven Systemzustands (*if (systemActive)*) wird alle 5 Sekunden überprüft, ob neue Sensorwerte vorliegen. Dies geschieht über die Funktion *scd41.getDataReadyFlag(ready)*. Ist die Variable *ready == true*, erfolgt das Auslesen der physikalischen Werte für Kohlenstoffdioxid, Temperatur und Luftfeuchtigkeit über die zentrale Funktion:

```
error = scd41.readMeasurement(co2, temp, hum);
```

Weitergabe der Daten:

Die Rohdaten werden in den Variablen *uint16_t co2*, *float temp* und *float hum* gespeichert, für das serielle Monitoring formatiert und anschließend über die Instanz *client.publish()* atomar an das MQTT-Netzwerk übermittelt.

4.1.3. Präsenzradar (LD2410C)

Präsenzanalyse: Die Bestimmung der Anwesenheit erfolgt über die logische ODER-Verknüpfung zweier geräteinterner Filterfunktionen:

```
bool menschPraesenz = radar.movingTargetDetected() ||  
                      radar.stationaryTargetDetected();
```

Hierdurch wird sichergestellt, dass sowohl sich bewegende Personen (*movingTargetDetected*) als auch vollkommen statisch sitzende Personen (*stationaryTargetDetected*) softwareseitig erfasst werden.

Zustandsmaschine für Energiesparmodus:

Erkennt der Sensor eine Präsenz, wird der Zeitstempel *lastPresenceTime* = *now* permanent aktualisiert. Befindet sich das System im Ruhezustand, schaltet *systemActive* = *true* die Ausleseschleife des Klimasensors wieder ein. Verlassen Personen den Raum, greift die zeitgesteuerte Differenzanalyse:

```
if (systemActive && (now - lastPresenceTime > TIMEOUT_SPARMODUS))
```

Überschreitet die Zeitspanne den Schwellenwert von *60.000ms* (*TIMEOUT_SPARMODUS*), wechselt das System zurück in den Energiesparmodus (*systemActive* = *false*), schaltet die Datenübertragung auf ein Intervall von 15 Sekunden um und deaktiviert die physische Peripherie.

4.1.4. LED-Statusampeln

Steuerungsfunktion *steuernLEDs(uint16_t wertCO2)*:

Diese Funktion implementiert die dreistufige Schwellenwert-Logik basierend auf dem übergebenen Parameter *wertCO2*. Sie steuert die Pins exklusiv mittels *digitalWrite()*, sodass zu jedem Zeitpunkt immer nur genau eine LED ein High-Signal erhält:

- **Stufe 1 (Gesunde Raumlufte):** Bei einem Wert von *wertCO2* < 1000 wird *ledGrün* auf *HIGH* gesetzt, während *ledGelb* und *ledRot* auf *LOW* geschaltet werden.
- **Stufe 2 (Erhöhte Aufmerksamkeit):** Bei Werten von *wertCO2* >= 1000 && *wertCO2* <= 2000 schaltet das System exklusiv *ledGelb* auf *HIGH*.
- **Stufe 3 (Kritischer Belüftungsbedarf):** Steigt der Wert über den kritischen Grenzwert von *2000ppm*, greift der *else*-Zweig. Der Pin *ledRot* erhält ein *HIGH-Signal*, um den Nutzer visuell zum Öffnen der Fenster aufzufordern.

CO2-Wertebereich	Leuchtende LED
0 – 1000 ppm	Grüne LED
1001 – 2000 ppm	Gelbe LED
Ab 2001 ppm	Rote LED

Deaktivierungsfunktion *offLEDs()*:

Um im Energiesparmodus eine unnötige Lichtbelästigung und Stromverbrauch zu vermeiden, wird diese Funktion aufgerufen, sobald das Radar-Timeout abgelaufen ist. Sie setzt alle drei LED-Pins mittels *digitalWrite(..., LOW)* zeitgleich auf ein Low-Potential und schaltet die Ampel vollständig ab.

4.2. Python-Backend

Das Backend des AeroGuard-Systems fungiert als zentrale Middleware. Es verarbeitet asynchron die eintreffenden Sensordaten des ESP32, persistiert die bereinigten Datensätze in einer relationalen SQLite Datenbank, betreibt die mathematische Berechnungs-Engine für den Luftqualitätsindex (IAQ) sowie die prädiktive KI-Trendanalyse und stellt dem Web-Frontend eine performante REST-API zur Verfügung.

4.2.1. Mathematische Berechnung-Engines

4.2.1.1. Algorithmus zur Bestimmung des Luftqualitätsindex (IAQ-Score)

a) Validierung der CO₂-Grenzwerte basierend auf medizinischen Richtlinien

Die Festlegung der Schwellenwerte für die Kohlenstoffdioxid-Konzentration im AeroGuard-Algorithmus stützt sich auf international anerkannte umweltmedizinische Richtlinien und epidemiologische Studien zur Innenraumlufthygiene. Als primäre Validierungsquellen dienen die umfassende Metastudie „*Carbon dioxide guidelines for indoor air quality: a review*“ (Abbildung 8) [1], sowie die offiziellen Richtwerte der Ad-hoc-Arbeitsgruppe Innenraumluft (Abbildung 9), herausgegeben vom deutschen Umweltbundesamt (UBA) [2].

Fig. 2: Eight identified CO₂ guidelines set with multi-tier limits, by country or organization.

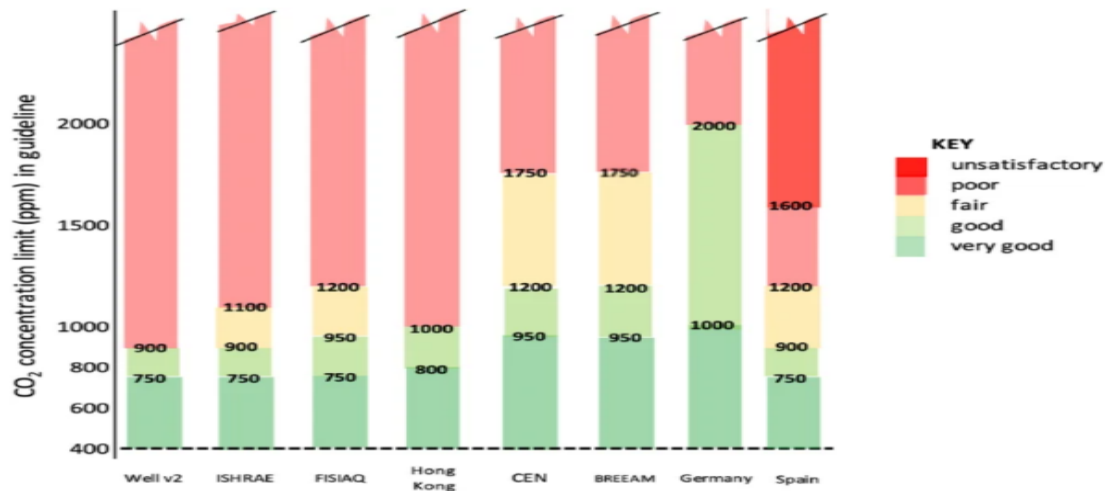


Abbildung 10: Richtlinien für Kohlendioxid in der Raumluftqualität

Gemäß diesen wissenschaftlichen Publikationen lassen sich die im Algorithmus verwendeten Intervalle epidemiologisch wie folgt begründen:

- **CO₂-Werte ≤ 1000ppm:** Dieser Bereich gilt wissenschaftlich als absolut unbedenklich. Das Umweltbundesamt definiert Werte unter 1000ppm als hygienisch akzeptabel.
- **CO₂-Werte zwischen 1000 - 2000ppm:** Dies stellt den Übergangsbereich dar. Obwohl die Luftqualität laut UBA-Leitfaden noch als akzeptabel gilt, setzt ab 1000ppm bereits eine statistisch nachweisbare Verringerung der Konzentrationsfähigkeit ein.
- **CO₂-Werte > 2000ppm:** Das UBA (2008) klassifiziert Werte über 2000ppm als „hygienisch inakzeptabel“ und fordert eine sofortige Belüftung. Laut der im *Journal of Exposure Science* veröffentlichten Review-Arbeit ist treten in diesem Bereich Symptome wie Müdigkeit, Kopfschmerzen und ein erhöhtes CO₂-induziertes Schläfrigkeitsgefühl häufig auf.

Tabelle 3

Klassifizierung der Raumluftqualität nach DIN EN 13779: 2007–09 (DIN 2007–09). Die Tabelle enthält in den Spalten 1–3 und 5 die Vorgaben der DIN EN 13779. Spalte 4 stellt beispielhaft für eine CO₂-Außenluftkonzentration von 400 ppm absolute CO₂-Konzentrationen in der Innenraumluft vor

Raumluft-Kategorie (Indoor Air)	Beschreibung	Erhöhung der CO ₂ -Konzentration gegenüber der Außenluft [ppm]	Absolute CO ₂ -Konzentration in der Innenraumluft [ppm]	Lüftungsrate/Außenluft- volumenstrom [l/s Person] [(m ³ /h Person)]
IDA 1	Hohe Raumluftqualität	≤ 400	≤ 800	> 15 (> 54)
IDA 2	Mittlere Raumluftqualität	> 400–600	> 800–1000	10–15 (> 36–54)
IDA 3	Mäßige Raumluftqualität	> 600–1000	> 1000–1400	6–10 (> 22–36)
IDA 4	Niedrige Raumluftqualität	> 1000	> 1400	< 6 (< 22)

Abbildung 11: Bekanntmachung des Umweltbundesamtes bezüglich Indoor Air

b) Berechnung des IAQ-Scores basierend auf den Umweltparametern (CO₂, Temperatur & Luftfeuchtigkeit)

Das Aeroguard-System normiert den Raumluftqualitäts-Index (IAQ-Score) auf einer Skala von **0 bis 100**.

Zur Zusammenführung der drei unterschiedlichen Umweltparameter (CO₂, Temperatur, relative Luftfeuchtigkeit) zu einem einzigen, normierten Index wird die **Methode der gewichteten Summe** (Weighted Linear Combination) angewendet. Diese Methode ist in der mathematischen Entscheidungstheorie und der multikriteriellen Analyse (MCDA) als valides Verfahren etabliert, um heterogene Kriterien auf eine einheitliche Skala (0 bis 100) zu projizieren.

Die mathematische Formulierung lautet:

$$IAQ_{\text{final}} = \sum_{i=1}^n w_i \cdot S_i$$

Mit:

$$\sum_{i=1}^n w_i \cdot S_i = (S_{CO_2} \cdot w_{CO_2}) + (S_{Temp} \cdot w_{Temp}) + (S_{Hum} \cdot w_{Hum})$$

Sco2: CO2-Score

Shum: Score der Luftfeuchtigkeit

Stemp: Temperatur-Score

Wco2: Gewichtung der CO2 = 0.5

Whum: Gewichtung der Feuchtigkeit = 0.25

Wtemp: Gewichtung der Temperatur = 0.25

4.2.1.2. Prädiktionsmodell (KI-Prognose)

Um ein Umkippen des Raumklimas frühzeitig zu prognostizieren, berechnet die Funktion *calculate_co2_trend(current_co2)* die mathematische Steigung (Trend) des CO2-Graphen. Das System speichert dazu die letzten 10 Messwerte im Array *co2_history* (Ringpuffer-Prinzip).

Rauschfilterung durch mathematische Cluster:

Um kurzfristige Messfehler oder Sensorrauschen zu eliminieren, appliziert der Algorithmus ein statistisches Cluster-Verfahren. Er bildet die Mittelwerte der ersten drei Elemente (*avg_start*) und der letzten drei Elemente (*avg_end*) des Puffers. Die Steigung *m* pro Minute berechnet sich aus der mathematischen Differenz der Mittelwerte geteilt durch das Zeitintervall *Delta* (*time_diff*):

$$m = \frac{\text{avg_end} - \text{avg_start}}{\Delta t}$$

Prädiktion des Alarmzeitpunkts:

Die mathematisch verbleibende Zeit (*minutes_to_alert*) bis zum Erreichen des kritischen Schwellenwerts von 2000ppm wird wie folgt berechnet:

$$\text{minutes_to_alert} = \frac{(1000 - \text{current_co2}) - \text{avg_start}}{m}$$

Sonderfälle: Ist die Steigung negativ ($m \leq 0$), Luftqualität verbessert sich) oder die Zeitdifferenz Null, gibt die Funktion -1 zurück (kein Alarm). Ist der Grenzwert bereits überschritten ($remaining_ppm \leq 0$), wird sofort 0 zurückgegeben. Alle gültigen mathematischen Ergebnisse werden kaufmännisch gerundet ($round()$) als ganzzahlige Minuten an das Frontend übergeben.

4.2.2. Raumbellegungsplan

Die Verwaltung des Raumbellegungsplans im Backend ist so konzipiert, dass sie eine dynamische und persistente Speicherung von Kurszeiten oder Raumnutzungsfenstern ermöglicht. Das System nutzt hierfür die relationale SQLite-Tabelle `occupations` und stellt über FastAPI eine vollständige Schnittstelle für administrative Datenänderungen bereit

Bevor Daten in die Datenbank geschrieben werden, validiert das Backend die eintreffenden HTTP-Anfragen mithilfe eines `class Occupation(BaseModel)`. Dieses Modell erzwingt eine strikte Typisierung der Felder `Day`, `start_time`, `end_time` und `Veranstaltungsname`.

Folgende REST-Routen und SQL-Transaktionen werden zu diesem Zweck benutzt: Auslesen von Daten (`GET /api/occupations`) - Löschen von Daten (`DELETE /api/occupations/{occ_id}`).

4.2.2. Historischen Graphen

Das Backend übernimmt hierbei die kritische Aufgabe der Datenfilterung, Begrenzung und chronologischen Sortierung.

- Zeitliche Eingrenzung und Speicherabfrage (`GET /api/history`): Ein unbegrenztes Auslesen der Tabelle `measurements` würde bei langen Laufzeiten den Arbeitsspeicher des Servers überlasten. Daher implementiert die Funktion `get_history()` eine strikte Mengengrenzung mittel des SQL-Befehls `LIMIT 12`.
- **Daten-Konvertierung und Zeit-Formatierung:** Das standardmäßige SQLite-Zeitstempelformat (`YYYY-MM-DD HH:MM:SS`) enthält für eine grafische Achsenbeschriftung zu viele redundante Informationen

4.3. Web-Dashboard

4.3.1. Technologie-Stack

Für die Entwicklung der Benutzeroberfläche von AeroGuard wurde eine moderne Architektur auf Basis des Frameworks **React.js** gewählt. Diese Entscheidung begründet sich durch die Notwendigkeit einer dynamischen Schnittstelle, die in der Lage ist, die Sensordaten in Echtzeit zu aktualisieren, ohne die Seite neu laden zu müssen.

- **Vite.js:** Wird als Build-Tool aufgrund seiner sehr schnellen Kompilierung und seines leistungsstarken Entwicklungsservers eingesetzt.
- **Tailwind CSS:** Das für das Design verwendete Framework ermöglicht durch einen Utility-First-Klassenansatz eine reaktive („*Responsive*“) Benutzeroberfläche, die sowohl für Mobilgeräte als auch für Computer optimiert ist.
- **Lucide-React:** Eine Icon-Bibliothek für eine intuitive visuelle Symbolisierung der Sensoren.
- **Recharts:** Eine spezialisierte Bibliothek zur Datenvisualisierung, die zur Generierung der historischen Verlaufsdiagramme verwendet wird.

4.3.2. Architektur der Komponenten

Die Benutzeroberfläche ist in modulare Komponenten unterteilt, um die Wartbarkeit des Codes zu fördern:

- **Header:** Zeigt den Verbindungsstatus des Systems an („*In Echtzeit*“).
- **StatCards:** Vier Module zur Anzeige der aktuellen Messwerte (CO₂, Temperatur, Luftfeuchtigkeit, IAQ-Score).
- **Meldungssystem:** Eine bedingte Komponente, die den CO₂-Schwellenwert (Grenze bei 2000ppm) analysiert, um einen visuellen Alarm auszulösen.

- **Historik-Graph:** Ein Liniendiagramm, das die Trends der letzten 24 Stunden anzeigt.
- **Belegungsplaner:** Ein Modul, das es dem Nutzer ermöglicht, den Raumbellegungsplan einzugeben.

4.3.3. Implementierung der Kernfunktionalitäten

a) Verwaltung der Alarme

Das System integriert eine aktive Überwachungslogik. Wenn der CO₂-Wert den Sicherheitsgrenzwert überschreitet, schaltet die Benutzeroberfläche dynamisch um, indem eine CSS-Animation (*animate-pulse*) auf einem roten Banner aktiviert wird. Ziel dabei ist, die Aufmerksamkeit des Nutzers sofort auf sich zu ziehen, um ihn zum Lüften aufzufordern.

b) Graphen Visualisierung

Das Recharts-Diagramm zeigt gleichzeitig vier Variablen an. Um die Lesbarkeit zu gewährleisten, wurden standardisierte Farbcodes verwendet:

- **Rot:** CO₂(Gefahr/Alarm).
- **Grün:** IAQ-Score (Gesamtgesundheit)
- **Blau / Gestrichelt:** Temperatur.
- **Gelb:** Luftfeuchtigkeit.

c) KI-Prognose

Das Dashboard empfängt eine berechnete Restzeit bis zum Erreichen des CO₂-Grenzwerts (2000ppm). Basierend auf dieser Restzeit aktualisiert das Dashboard die Anzeige in Echtzeit und gibt entweder eine Entwarnung („Luft stabil“), eine Wartezeit in Minuten bis zum Erreichen des Grenzwerts oder eine Sofortaufforderung („Sofort lüften!“) aus.

4.3.4. Technische Besonderheiten des Codes

Der Quellcode des Frontends basiert nicht auf Standard-JavaScript, sondern auf einer fortschrittlicheren Architektur:

- **JSX-Syntax (JavaScript XML):** JSX wurde verwendet, um die Programmierlogik und die Struktur der Benutzeroberfläche zu verschmelzen. Dies ermöglicht es der Komponente, die Anzeige (wie

den Wechsel von Grün zu Rot beim CO2-Alarm) extrem schnell neu zu berechnen, ohne dass DOM manuell manipulieren zu müssen.

- **JavaScript ES6+:** Die Verwendung von Pfeilfunktionen (*Arrow Functions*), Modulen (*Import/Export*) und „States“ (*Hooks*) garantiert einen sauberen, modularen und leicht zu wartenden Code.
- **Transpiliationsprozess:** Obwohl der Code aus Gründen des Entwicklungskomforts in JSX geschrieben ist, wird er vom Tool *Vite* in reines JavaScript (*Vanilla JS*) „transpiliert“, damit er von allen modernen Webbrowsern perfekt interpretiert werden kann.

5. Problemen und deren Lösungen

5.1. Kommunikations- und Konfigurationskonflikte beim MQTT-Broker

- **Problem:** Bei der ersten Inbetriebnahme des Systems schlug die Datenübertragung zwischen dem ESP32 und dem Python-Backend fehl. Der MQTT-Broker (*Eclipse Mosquitto*) blockierte die Pakete, da die Standardkonfiguration moderner Broker aus Sicherheitsgründen keine anonymen Verbindungen von externen IP-Adressen (wie der des ESP32 im lokalen WLAN) außerhalb des localhost erlaubt.
- **Lösung:** Die Konfigurationsdatei des Mosquitto-Brokers (*mosquitto.conf*) wurde manuell angepasst und optimiert. Durch das explizite Setzen der Parameter *listener 1883 0.0.0.0* (Bindung an alle Netzwerkschnittstellen) und *allow_anonymous true* wurde die Kommunikationsbrücke freigeschaltet, sodass der ESP32 seine Daten fehlerfrei an das Backend publizieren konnte.

5.2. Spannungsmodelle & Instabilität bei der Sensor-Parallelversorgung

- **Problem:** Bei der direkten Stromversorgung mehrerer Sensoren über die Pins des ESP32 kam es zu Spannungsabfällen. Da der photoakustische CO2-Sensor (SCD41) und das Präsenzradar (LD2410C) temporär hohe Stromspitzen aufweisen, brach die Versorgungsspannung ein. Dies führte zu fehlerhaften Sensormessungen und sporadischen Abstürzen des ESP32.

- **Lösung:** Zur Stabilisierung der Spannungsversorgung wurde das **Breadboard als zentraler Bus-Verteiler** genutzt. Die Spannungsquelle (5V bzw. 3,3V) des ESP32 wurde direkt auf die rote Hauptstromschiene des Breadboards gelegt. Von dort aus wurden die Sensoren parallel mit Energie versorgt. Diese sternförmige Verteilung verringerte die Übergangswiderstände, verblockte die Stromspitzen und garantierte eine konstant stabile Betriebsspannung für alle Komponenten.

5.3. Synchronisationsfehler zwischen Frontend-Eingabe & SQLite-Datenbank

- **Problem:** Beim Hinzufügen eines neuen Kurses über das visuelle Formular des React-Frontends wurde dieser zwar lokal in der UI angezeigt, jedoch nicht dauerhaft in der SQLite-Datenbank gespeichert. Nach einem Seitenaufruf war der Kurs gelöscht. Ursache war eine fehlerhafte JSON-Datenstruktur im HTTP-Request-Body, die vom Pydantic-Modell des Backends aufgrund von Formatabweichungen blockiert wurde.
- **Lösung:** Die Kommunikationsschnittstelle wurde harmonisiert. Im Frontend wurde die Funktion *add_occupation* so angepasst, dass die Formulardaten exakt sequenziell im JSON-Format serialisiert und mit dem Header *'Content-Type': 'application/json'* übermittelt werden. Auf der Backend-Seite wurde die Route *@app.post("/api/occupations")* so robust programmiert, dass sie das Pydantic-Modell (*Occupation*) validiert, die Werte entpackt und mittels einer geschützten SQL-Transaktion dauerhaft in die Tabelle *occupations* einträgt.

6. Inbetriebnahme & Deployment (Vom Quellcode zur Laufenden Umgebung)

Dieses Kapitel beschreibt die exakte Schritt-für-Schritt-Prozedur, um das AeroGuard-Gesamtsystem aus dem Quellcode heraus zu installieren, zu konfigurieren und in eine aktive Ausführungsumgebung zu überführen. Das System teilt sich in drei Hauptkomponenten auf: die Hardware-Firmware, das Python-Backend und das React-Web-Frontend.

6.1. Schritt 1: Starten der Infrastruktur (MQTT-Broker)

Bevor die Softwarekomponenten kommunizieren können, muss die Nachrichten-Infrastruktur im lokalen Netzwerk bereitgestellt werden.

- **Installation:** Falls noch nicht geschehen, muss der MQTT-Broker *Eclipse Mosquitto* auf dem Zielrechner (Server/PC) installiert werden.
- **Konfiguration:** Die Datei *mosquitto.conf* öffnen und die in Kapitel 5.1 erarbeiteten Zeilen einfügen, um externe Verbindungen des ESP32 zuzulassen:

```
listener 1883 0.0.0.0
allow_anonymous true
```

- **Start des Dienstes:** Den Broker über das Terminal oder die Systemdienste starten:

unter Windows: net start mosquitto

unter Linux/macOS: sudo systemctl start mosquitto

6.2. Schritt 2: Einrichtung und Start des Backends (FastAPI & SQLite)

Das Backend verwaltet die Datenbank und stellt die REST-Schnittstellen bereit. Es benötigt eine Python-Umgebung.

- **Virtuelle Umgebung erstellen & aktivieren:** Im Stammverzeichnis des Backend-Quellcodes ein Terminal öffnen und eine isolierte Umgebung einrichten:

```
python -m venv venv
```

unter Windows:

```
venv\Scripts\activate
```

unter Linux/macOS:

```
source venv/bin/activate
```

- **Abhängigkeiten installieren:** Die benötigten Bibliotheken über den Paketmanager pip installieren:

```
pip install fastapi uvicorn paho-mqtt pydantic
```

- **Datenbank initialisieren:** Beim ersten Start des Quellcodes prüft das Python-Skript automatisch, ob die Datei *aeroguard.db* existiert. Falls nicht, werden die Tabellen *measurements* und *occupations* über das integrierte SQLite-Modul autonom angelegt.
- **Server starten:** Der FastAPI-Server wird über den ASGI-Server *Uvicorn* auf Port 8000 gestartet:

uvicorn main:app --reload --host 127.0.0.1 --port 8000

Das Backend läuft nun aktiv und wartet auf MQTT-Daten sowie HTTP-Anfragen.

6.3. Schritt 3: Kompilierung und Start des Web-Frontends (React.js / Vite)

Das Frontend bereitet die Daten visuell im Browser auf und läuft isoliert auf Node.js-Basis.

- **Abhängigkeiten auflösen:** Im Verzeichnis des Frontend-Quellcodes ein Terminal öffnen und die Node-Pakete (inklusive *lucide-react* und *recharts*) über den Node Package Manager installieren:

npm install

- **Entwicklungsserver starten:** Das Projekt wird über das Build-Tool *Vite* kompiliert und lokal bereitgestellt:

npm run dev

- **Aufruf der GUI:** Das Terminal gibt nach erfolgreicher Kompilierung die lokale URL aus (*standardmäßig http://localhost:5173 oder http://localhost:3000*). Diese wird im Webbrowser geöffnet, um das interaktive Dashboard anzuzeigen.

6.4. Schritt 4: Flashen der Firmware auf den ESP32-Mikrocontroller

Damit die Hardwaredaten gesammelt werden, muss der C++ Quellcode auf den Mikrocontroller übertragen werden.

- **Entwicklungsumgebung öffnen:** Den Firmware-Ordner in der *Arduino IDE* oder in *VS-Code (mit PlatformIO)* öffnen.
- **Netzwerkparameter anpassen:** Im Quellcode die Variablen für die lokalen WLAN-Zugangsdaten sowie die IP-Adresse des in Schritt 1 gestarteten MQTT-Brokers eintragen:

```
const char* ssid = "DEIN_WLAN_NAME";  
const char* password = "DEIN_WLAN_PASSWORT";  
const char* mqtt_server = "IP_DEINES_PC";
```

- **Kompilieren und Hochladen:** Den ESP32 über ein USB-C-Kabel an den PC anschließen, den korrekten COM-Port auswählen und auf „Hochladen“ (*Upload*) klicken.

Nachdem alle vier Schritte erfolgreich abgeschlossen sind, befindet sich das Gesamtsystem in der laufenden Umgebung: Der ESP32 sendet die gemessenen Werte via MQTT an den Broker, das Backend verarbeitet und speichert sie in der SQLite-Datenbank, und das React-Frontend spiegelt den aktuellen Raumstatus sowie die berechneten KI-Prognose-Minuten im Web-Dashboard wider.

7. Fazit & Ausblick

7.1. Fazit

Im Rahmen dieses Projekts wurde das AeroGuard-System erfolgreich von der theoretischen Konzeption bis zu einem einsatzbereiten, vernetzten Prototyp realisiert. Alle in der Zielsetzung definierten Kernfunktionalitäten wurden vollständig implementiert und erfolgreich getestet.

Besonders hervorzuheben ist das gelungene Zusammenspiel der heterogenen Systemkomponenten: Der ESP32 agiert als stabile Edge-Schnittstelle, die durch die Timer-Integration und die Integration des Präsenzradars eine fehlerfreie Präsenzerkennung und Datenerfassung

garantiert. Die Übertragung via MQTT erwies sich auch bei der sequenziellen Partitionierung der Nachrichten als hocheffizient, da serverseitig ein atomarer Puffer-Mechanismus die Datenintegrität in der SQLite-Datenbank sichert.

Durch die mathematische Justierung des IAQ-Scores und die Implementierung der KI-Trendprognose im Backend liefert das System einen echten funktionalen Mehrwert gegenüber marktüblichen Standard-Messgeräten. Trotz der hardwareseitigen Einschränkung beim Gehäusebau (Karton-Prototyp statt 3D-Druck) konnte ein thermodynamisch stabiles System geschaffen werden, das durch gezielte Freischnitte präzise Messwerte liefert. Die reaktive React-Oberfläche rundet das Projekt ab, indem sie dem Endnutzer komplexe Datenströme in eine intuitive, leicht verständliche visuelle Handlungsaufforderung übersetzt. AeroGuard beweist eindrucksvoll, wie mit kostengünstigen Hardware-Komponenten und moderner Software-Architektur ein intelligentes IoT-System zur Gesundheitsprävention realisiert werden kann.

7.2. Ausblick

Obwohl der aktuelle Prototyp stabil und zuverlässig arbeitet, bietet das AeroGuard-System Raum für zukünftige funktionale und technologische Erweiterungen:

- **Optimierung des Gehäuses (Additive Fertigung):** Sobald die Ressourcen verfügbar sind, soll das handgefertigte Kartongehäuse durch ein passgenaues, CAD-konstruiertes und 3D-gedrucktes Gehäuse aus thermisch stabilem Kunststoff (z. B. PETG oder ABS) ersetzt werden. Dies erhöht die Langlebigkeit und die visuelle Professionalität des Geräts.
- **Ausbau des prädiktiven Algorithmus (Echte KI):** Das aktuelle lineare Regressionsmodell zur Berechnung der `prediction_minutes` kann durch fortgeschrittene Machine-Learning-Modelle (z. B. ein im Backend trainiertes *LSTM-Netzwerk* / Long Short-Term Memory) ersetzt werden. Dieses könnte zusätzlich den historischen Raumbelungsplan sowie externe Wetterdaten (Außentemperatur und Windgeschwindigkeiten) einbeziehen, um die Prognosegenauigkeit bei unregelmäßigen Raumbelungen weiter zu verfeinern.
- **Aktive Aktuatorik (Smart-Home-Integration):** In einer nächsten Ausbaustufe könnte das System von einer rein informierenden Ampel zu einem aktiven Steuerungssystem erweitert werden. Über smarte Relais oder WLAN-Steckdosen (z. B. via Home

Assistant-Anbindung) könnten automatische Fensteröffner oder Lüftungsanlagen direkt angesteuert werden, sobald das System einen kritischen Wert prognostiziert.

8. Stundenliste

Phasen	Tätigkeiten	Ist-Zeit	Soll-Zeit
Phase 1: Frontend Basis & Visualisierung	Setup Installation (Vite & Tailwind) für Frontend Design + Erstellung der groben Frontend-Model & -Architektur	6 Std.	8 Std.
	Statcards-Design, Recharts-Integration + Graphen-Erstellung & -Verwaltung	8 Std.	10 Std.
	Implementierung der Ein- & Ausgabemaske des Raumbelegungsplans	6 Std.	10 Std.
	Implementierung der Anzeige der wertbedingten Warnmeldung/KI-Prognose	2 Std.	4 Std.
Phase 2: Infrastruktur & Backend	API-Installation & -Anbindung, Erstellung & Strukturierung der SQLite-Datenbank für Belegungsdaten	4 Std.	7 Std.
	Implementierung & Test der CRUD-Operationen	3 Std.	4 Std.
	Entwicklung & Test der Logik der prädiktiven Lüftungsvorhersage	5 Std.	8 Std.

	Algorithmus für IAQ-Score	20 Std.	25 Std.
	Implementierung & Test der dynamischen Speicherung von Raumlufthdaten und deren Visualisierung auf dem Dashboard	4 Std.	5 Std.
Phase 3: Hardware - Integration	Implementierung des Dev-Modules des ESP32 auf Arduino IDE	8 Std.	5 Std.
	Einsatz der SCD41-Sensor & Kalibrierung	7 Std.	3 Std.
	Einsatz des Mosquitto Brouker & WLAN-Verbindung	2 Std.	1 Std.
	Erweiterung des Python-Backend, Anpassung des ESP32-Dev-Moduls zur Auslesung des Sensordaten	5 Std.	4 Std.
	KI-Prognose	4 Std.	6 Std.
	Aufbau des gesamten Systems & Gehäuse	3 Std.	4 Std.
	Integration & Tests	7 Std.	10 Std.
Phase 4: Verfeinerung & Dokumentation	Bearbeitung des IAQ-Algorithmus	5 Std.	/
	Erfassung der Dokumentation	10 Std.	/

	Bearbeitung der Logik der historischen Graphen auf den Frontend	4 Std.	/
--	---	--------	---

Quellen:

[1]: [Carbon dioxide guidelines for indoor air quality: a review | Journal of Exposure Science & Environmental Epidemiology](#)

[2]: [kohlendioxid 2008.pdf](#)